

Paterni Ponašanja

Strategy patern

Opis: Strategy patern omogućava klijentu izbor jednog od algoritama iz familije algoritama. Implementacije algoritama su nezavisne od klijenata koji ih koriste.

Primjena: U našem sistemu, ovaj patern bismo mogli iskoristiti kod klase PlanTreninga.

Obrazloženje: Algoritmi za preporuku plana treninga se razlikuju zavisno od FitnessCilj-a korisnika (npr. mršavljenje, mišićna masa, održavanje kondicije). Da PlanTreninga (ili servis koji ga generiše) ne bi imao ogromne if-else blokove za provjeru cilja, za svaki način kreiranja plana bismo mogli napraviti posebnu klasu strategije.

Prednost: Time se omogućava lako dodavanje novih ciljeva i algoritama treninga bez potrebe za izmjenom osnovnih klasa.

Način implementacije: S obzirom da korisnik može mijenjati svoj cilj i time zahtijevati novi algoritam, Method injection ili Constructor injection su optimalni za naš sistem.

State patern

Opis: State patern omogućava objektu da izmijeni svoje ponašanje na osnovu promjene njegovog unutrašnjeg stanja. Umjesto velikog broja provjera, svako stanje postaje posebna klasa.

Primjena: Najbolje je primijeniti na domenske objekte poput Clanarina i Rezervacija.

Obrazloženje: Clanarina se ponaša drugačije zavisno od svog stanja (definisanog npr. kroz StatusClanarine - Aktivna, Istekla, Suspendovana). Evidencija Dolazak klase nad aktivnom članarinom treba dozvoliti prolaz korisniku, dok nad isteklom blokira ulaz.

Prednost: Pravimo interfejs StanjeClanarine i konkretne klase. Sistem postaje fleksibilniji; novo stanje se dodaje kao nova klasa bez velikih izmjena koda.

Template Method patern

Opis: Omogućava izdvajanje određenih koraka algoritma u odvojene podklase. Struktura (kostur) algoritma se ne mijenja - izdvajaju se samo specifični dijelovi.

Primjena: Ovaj patern je idealno primijeniti na klasu Izvjestaj i generisanje različitih statistika zavisno od Tiplzvjestaja.

Obrazloženje: U baznoj klasi IzvjestajGenerator možemo definisati kostur kroz metodu KreirajIzvjestaj(), koja će pozivati korake tačnim redoslijedom (PrikupljanjePodataka, Formatiranje, Export).

Prednost: Konkretne podklase, kao što su IzvjestajDolazaka i IzvjestajFinansija, redefinišu samo one korake koji su jedinstveni za njih (npr. izvor podataka). Time se izbjegava dupliranje koda i forsira jedinstven tok kreiranja izvještaja.

Observer patern

Opis: Uspostavlja relaciju "jedan-na-više" između objekata, tako da kada jedan objekat promijeni stanje, svi prijavljeni objekti (posmatrači) automatski budu obaviješteni.

Primjena: Idealno za modul EmailObavjestenje.

Obrazloženje: Kada se npr. otkaže GrupniTrening ili istekne Clanarina, sistem treba obavijestiti korisnika. U ovom slučaju GrupniTrening predstavlja Subjekt, dok su klase za notifikaciju posmatrači.

Prednost: Objekat koji mijenja stanje samo objavljuje događaj (Event), a svi registrovani servisi (posmatrači) reaguju na taj događaj. Subjekt ne mora direktno znati koga sve obavještava.

Iterator patern

Opis: Omogućava pristup elementima kolekcije bez poznavanja interne strukture u kojoj su ti podaci smješteni.

Primjena: Prolazak kroz kompleksne liste poput liste objekata Termin ili Dolazak.

Obrazloženje: Termin u teretani mogu biti pohranjeni i organizovani po danima, trenerima, salama ili tipu treninga. Iterator kreira jedinstven algoritam navigacije redom kroz svaki Termin.

Prednost: U slučaju da se u budućnosti promijeni struktura podataka koja čuva termine unutar baze/memorije, ostatak sistema (UI i obrada) se ne mora mijenjati.

Visitor patern

Opis: Omogućava odvajanje operacija i algoritama od strukture samih objekata nad kojima se izvršavaju. Dodaju se nove funkcionalnosti postojećim klasama bez njihove modifikacije.

Primjena: Različite globalne operacije (poput obračuna ukupne dobiti, eksportovanja podataka ili generisanja specifičnih view modela) nad korisnicima ili treninzima.

Obrazloženje: Ako za Korisnik, Trener i GrupniTrening moramo dodati operaciju Export(), umjesto zagađivanja domenskih klasa kreiramo metodu Accept(Visitor v). Konkretni visitor (npr. DataExportVisitor) izvršava operaciju izvlačenja.

Prednost: Domenske klase ostaju fokusirane samo na svoje osnovne atribute (Single Responsibility), dok su globalne akcije izmještene u vanjsku strukturu (Visitora).

Command patern

Opis: Koristi se za enkapsuliranje svih informacija potrebnih za odgođeno izvođenje akcije. Operacija postaje samostalan objekat.

Primjena: Akcije koje korisnik poduzima nad entitetom Rezervacija (kreiranje, otkazivanje, ažuriranje).

Obrazloženje: Umjesto direktnog pozivanja metoda za rezervaciju unutar kontrolera, svaka akcija postaje klasa (npr. KreirajRezervacijuCommand).

Prednost: Objekat koji inicira akciju je apsolutno odvojen od logike njenog izvršavanja. Ovo pruža mogućnost čuvanja akcija u red čekanja, logovanja na nivou sistema, kao i lake implementacije opcije za poništavanje (undo) komandi, ukoliko član slučajno otkaže trening.